Computational Thinking

What is Computational Thinking?

Computational Thinking (CT) is a **problem-solving process** that involves breaking down complex problems systematically to develop solutions that a computer or human can execute. CT ultimately produces an **algorithm** – a precise, replicable series of steps.

Four Pillars of Computational Thinking

1. Decomposition

Definition

Breaking a **complex problem into smaller, more manageable sub-problems**, each of which can be solved individually.

Why it matters:

- Smaller parts are easier to understand and tackle individually
- Enables delegation and parallel teamwork
- Helps identify patterns and relationships among components
- Prioritises time allocation based on importance/urgency

Example – Creating an app: What it looks like, target audience, graphics, audio, navigation, testing, distribution.

Example – Solving a crime: What crime? When? Where? Evidence? Witnesses? Similar crimes?

2. Pattern Recognition

Definition

Identifying **similarities, differences, trends, or repeated structures** across decomposed sub-problems to apply common solutions efficiently.

Examples:

- Computing first n square numbers: each is $i \times i$ – same pattern, loop once
- Netflix recommendations, clickstream analysis, disease outbreak detection
- Machine Learning – clustering hidden patterns in data

3. Abstraction

Definition

Focusing on essential details and ignoring irrelevant information to simplify the problem representation.

Examples:

- A road map: shows roads, cities, and rail – omits road width, exact geography
- Rectangle area program: keeps width, height, formula; discards actual values
- Race-car game: simplified controls, polygonal models, infinite fuel

Levels of abstraction in computing: Hardware → OS → Language → Application

4. Algorithmic Thinking

Definition

Designing a **precise, step-by-step, replicable process** (algorithm) that accepts defined inputs and produces defined outputs.

Properties of a good algorithm:

- **Purposeful** – solves a specific problem
- **Describable** – can be communicated clearly
- **Replicable** – produces consistent results
- **Finite** – terminates in a finite number of steps

Three Steps of Computational Thinking in Practice

1. **Problem Specification**: Analyse and state the problem precisely; define criteria for the solution using decomposition, abstraction, and pattern recognition.
2. **Algorithmic Expression**: Design the algorithm with appropriate data representations using flowcharts, pseudocode, or diagrams.
3. **Solution Implementation & Evaluation**: Implement, test, debug, and check if the solution generalises to related problems.

Programming Paradigms (Overview)

Paradigm	Focus	Examples
Imperative	Step-by-step *how*	C, Java
Declarative	*What* result is needed	SQL, Haskell
Procedural	Modular functions/procedures	C, Pascal
Object-Oriented	Objects & classes	Java, Python, C++
Functional	Mathematical functions, immutable	Haskell, Lisp
Logic	Rules and facts	Prolog
Event-Driven	Responds to events (clicks, keys)	JavaScript

Search Algorithms

Sequential (Linear) Search

Algorithm

Starting from the **first element**, compare the target x with each element one by one until found or the list is exhausted.

Pseudocode:

```
seqsearch(n, S[], x, &location):
  location = 1
  while (location <= n AND S[location] != x):
    location++
  if (location > n):
    location = 0   // not found
```

Case	Comparisons	Complexity
Best Case	1 (target at index 1)	$O(1)$
Average Case	$(n+1)/2$	$O(n)$
Worst Case	n (target last or not present)	$O(n)$

Key point: No pre-sorting needed. Works on any list.

Binary Search

Algorithm – Requires Sorted Array

Repeatedly **halve the search range**: compare target with the midpoint, then search left or right half accordingly.

Pseudocode:

```
binsearch(n, S[], x, &location):
  low = 1; high = n; location = 0
  while (low <= high AND location == 0):
    mid = (low + high) / 2   // integer division
    if x == S[mid]:
      location = mid
    else if x < S[mid]:
      high = mid - 1
    else:
      low = mid + 1
```

Case	Comparisons	Complexity
Best Case	1 (target is the midpoint)	$O(1)$
Worst Case	$\lfloor \log_2 n \rfloor + 1$	$O(\log n)$

Worked Example – Find 176 in `[44, 56, 78, 89, 105, 132, 176, 210, 250, 305]`:

Step	low	high	mid	Action
1	1	10	5	$S[5]{=}105 < 176 \Rightarrow$ low $= 6$
2	6	10	8	$S[8]{=}210 > 176 \Rightarrow$ high $= 7$
3	6	7	6	$S[6]{=}132 < 176 \Rightarrow$ low $= 7$
4	7	7	7	$S[7]{=}176 = 176 \Rightarrow$ **Found!**

4 comparisons versus 8 for sequential search.

Binary vs Sequential – Key Comparison

	Sequential	Binary
Requires sorted data	No	**Yes**
Worst case	$O(n)$	$O(\log n)$
Best case	$O(1)$	$O(1)$
Use when	Data unsorted or small	Data sorted, large

Fibonacci Sequence

Definition

A sequence where each term is the sum of the two preceding terms:

$$F_0 = 0, \quad F_1 = 1, \quad F_n = F_{n-1} + F_{n-2} \quad \text{for } n \geq 2$$

First 15 terms: 0, 1, 1, 2, 3, 5, 8, 13, 21, 34, 55, 89, 144, 233, 377

Recursive Implementation

```
int fib(int n):
  if n <= 1: return n
  else: return fib(n-1) + fib(n-2)
```

Complexity: $O(2^n)$ – exponential, inefficient due to repeated subproblems.

Iterative Implementation

```
int fib2(int n):
  f[0] = 0
  if n > 0: f[1] = 1
  for i = 2 to n: f[i] = f[i-1] + f[i-2]
  return f[n]
```

Complexity: $O(n)$ – linear, much more efficient.

Useful Fibonacci Facts

- The ratio of consecutive Fibonacci numbers approaches the **Golden Ratio** $\varphi \approx 1.618$
- Tribonacci variant: each term is the sum of the *three* preceding terms
- Appears in nature: sunflower spirals, leaf arrangements, nautilus shells

Recursion

Definition

A function that **calls itself** with a smaller/simpler input until it reaches a **base case** (stopping condition).

Structure of Recursion

Every recursive implementation must have:

1. **Base Case** – the simplest instance that does not recurse (e.g. $n = 0$ or $n = 1$). Avoids infinite recursion.
2. **Recursive Case** – breaks the problem into a smaller sub-problem and calls itself.

Example – Factorial:

$$n! = n \times (n-1)! \quad \text{with} \quad 0! = 1$$

```
factorial(n):
  if n == 0: return 1
  else: return n * factorial(n-1)
```

Execution of `factorial(5)`: $5 \times 4 \times 3 \times 2 \times 1 = 120$

Types of Recursion

Summary of Recursion Types

Type	Description
Head Recursion	Recursive call is the *first* statement; work done on the way *back*
Tail Recursion	Recursive call is the *last* statement; work done *before* the call
Tree Recursion	Function calls itself *more than once* per invocation
Nested Recursion	Recursive call *passes a recursive call* as an argument
Direct Recursion	Function calls itself directly
Indirect Recursion	Function A calls B, which calls A – forms a cycle

Head Recursion

```
head(n):
  if n == 0: return
  head(n - 1)       // recursive call FIRST
  print(n)          // work done AFTER return
```

Output for `head(5)`: 1 2 3 4 5

Tail Recursion

```
tail(n):
  if n == 0: return
  print(n)          // work done FIRST
  tail(n - 1)       // recursive call LAST
```

Output for `tail(5)`: 5 4 3 2 1. **Can be optimised** by the compiler (no new stack frame needed).

Tree Recursion – Fibonacci Example

```
fib(n):
  if n <= 1: return n
  return fib(n-1) + fib(n-2)  // TWO recursive calls
```

Creates a tree of calls; many sub-problems recomputed $\Rightarrow O(2^n)$.

Tracing Recursive Functions

Trace Table Method

Track each function call in a table with columns: **Function Call** — **Parameter(s)** — **Return Value**

Example – double_factorial(8): (computes $n!! = n \times (n-2) \times \ldots$)

Call	n	Return Value
double_factorial(8)	8	8× double_factorial(6) $= 8 \times 6 \times 4 \times 2 \times 1 = 384$
double_factorial(6)	6	6× double_factorial(4)
double_factorial(4)	4	4× double_factorial(2)
double_factorial(2)	2	2× double_factorial(0)
double_factorial(0)	0	1 (*base case:* $n \leq 1$)

Final result: $8 \times 6 \times 4 \times 2 \times 1 = 384$

Trace Tree Method

Draw each recursive call as a node. Children are sub-calls. Leaf nodes are base cases. Return values propagate upward.

Example – sum_squares(3): (computes $1^2 + 2^2 + 3^2 = 14$)

$$\begin{aligned} \text{sum\_squares}(3) &\to 3^2 + \text{sum\_squares}(2) = 9 + 5 = 14 \\ \text{sum\_squares}(2) &\to 2^2 + \text{sum\_squares}(1) = 4 + 1 = 5 \\ \text{sum\_squares}(1) &\to 1^2 + \text{sum\_squares}(0) = 1 + 0 = 1 \\ \text{sum\_squares}(0) &\to 0 \quad \text{(base case)} \end{aligned}$$

The Call Stack

- Recursion is managed by the **call stack** (LIFO – Last In, First Out)
- Each call creates a new **stack frame** storing local variables and return address
- Deep recursion can cause a **stack overflow** error

Recursion vs Iteration

Aspect	Recursion	Iteration
Readability	Often clearer for tree/graph problems	Clearer for simple loops
Memory	Higher (call stack frames)	Lower
Performance	Can be slower (function-call overhead)	Usually faster
Risk	Stack overflow for deep recursion	None
Best for	Trees, graphs, divide & conquer, backtracking	Simple loops, counters

Naturally recursive problems: nested directory structures, parsing expressions, Tower of Hanoi, DFS in graphs, fractal generation.

Recursion Best Practices and Common Pitfalls

Best Practices

- Always define a clear and reachable base case
- Ensure each recursive call makes progress *towards* the base case
- Be mindful of memory – consider memoisation for overlapping sub-problems
- Test with small inputs first; trace manually

Common Pitfalls

- **Missing base case** – leads to infinite recursion
- **Stack overflow** – recursion too deep
- **Incorrect recursive arguments** – wrong progress toward base case
- **Redundant sub-problem recomputation** – use memoisation to fix

Applications of Recursion

- Searching and sorting (merge sort, quicksort, tree traversals)
- Mathematical sequences (Fibonacci, factorial, power)
- Compiler design (parsing)
- Computer graphics (fractals, Mandelbrot set)
- Artificial intelligence (recursive neural networks, backtracking)

Time Complexity and Asymptotic Notation

Why Analyse Algorithms?

Purpose

To determine an algorithm's **efficiency** in terms of time and space as a function of input size n, *independent* of hardware, OS, or programming language.

Basic Operation

Definition

The **basic operation** is the *dominant* operation whose count determines the algorithm's time complexity. It is the most frequently executed operation.

Examples:

- Searching: comparison ($A[i] == key$)
- Sorting: comparison ($A[i] > A[j]$)
- Matrix multiplication: multiplication ($A[i][k] \times B[k][j]$)
- Summing elements: addition ($sum = sum + A[i]$)

Input Size

The measure of problem size, e.g.:

- Number of elements in an array
- Length of a list
- Number of rows/columns in a matrix
- Number of vertices/edges in a graph

Three Cases of Analysis

Best, Average, and Worst Cases

Case	Notation	Meaning	Usage
Best	$B(n)$, Ω	Min operations over all inputs of size n	Theoretical lower bound
Average	$A(n)$, Θ	Expected operations over probable inputs	Realistic performance
Worst	$W(n)$, O	Max operations over all inputs of size n	**Most commonly used**

For Sequential Search:

- $B(n) = 1$ – target at position 1
- $A(n) = \frac{n+1}{2}$ – average when x is present
- $W(n) = n$ – target at end or not present

For Binary Search:

- $B(n) = 1$ – target is midpoint
- $W(n) = \lfloor \log_2 n \rfloor + 1$

Asymptotic Notations

Big-O (Upper Bound – Worst Case)

Formal Definition

$g(n) \in O(f(n))$ if there exist constants $c > 0$ and $N \geq 0$ such that:

$$g(n) \leq c \cdot f(n) \quad \text{for all } n \geq N$$

Meaning: $g(n)$ does not grow faster than $f(n)$ eventually.

Big-Ω (Lower Bound – Best Case)

Formal Definition

$g(n) \in \Omega(f(n))$ if there exist constants $c > 0$ and $N \geq 0$ such that:

$$g(n) \geq c \cdot f(n) \quad \text{for all } n \geq N$$

Meaning: $g(n)$ grows at least as fast as $f(n)$ eventually.

Big-Θ (Tight Bound – Average Case)

Formal Definition

$g(n) \in \Theta(f(n))$ if there exist constants $c, d > 0$ and $N \geq 0$ such that:

$$c \cdot f(n) \leq g(n) \leq d \cdot f(n) \quad \text{for all } n \geq N$$

Meaning: $g(n)$ grows at exactly the same rate as $f(n)$ eventually.

Small-o (Strict Upper Bound)

$g(n) \in o(f(n))$: g grows *strictly* slower than f. Analogous to $<$ (Big-O is like $\leq$).

Common Complexity Classes (Ordered from Best to Worst)

Complexity Hierarchy

$$O(1) \subset O(\log n) \subset O(n) \subset O(n \log n) \subset O(n^2) \subset O(n^3) \subset O(2^n) \subset O(n!)$$

Class	Name	Example	Notes
$O(1)$	Constant	Array index access	Independent of input size
$O(\log n)$	Logarithmic	Binary search	Halves problem each step
$O(n)$	Linear	Sequential search	One pass through data
$O(n \log n)$	Linearithmic	Merge sort, Heap sort	Divide & conquer sorts
$O(n^2)$	Quadratic	Bubble sort, two nested loops	Doubles squared when n doubles
$O(n^3)$	Cubic	Three nested loops, matrix multiply	Triples cubed
$O(2^n)$	Exponential	Recursive Fibonacci, brute-force	Very slow for large n

$O(n!)$	Factorial	Permutations, travelling salesman	Worst – even moderate n is intractable

Simplification Rules for Big-O

Rules

1. **Drop constants**: $5n \Rightarrow O(n), \quad 100 \Rightarrow O(1)$
2. **Drop lower-order terms**: $n^2 + n + 10 \Rightarrow O(n^2)$
3. **Keep only the fastest-growing term**
4. **Logs: base doesn't matter** in Big-O: $O(\log_2 n) = O(\log_{10} n) = O(\ln n)$

Rules for Computing Big-O from Code

Pattern	Rule	Example
Single statement	$O(1)$	`x = x + 1`
Sequential statements	Add, keep dominant	$O(1) + O(n) = O(n)$
Consecutive loops	Add	$O(n) + O(n) = O(n)$
Nested loops (fixed n)	Multiply	$O(n) \times O(n) = O(n^2)$
Loop halving/doubling i	$O(\log n)$	`i *= 2` or `i /= 2`
Three nested loops	Multiply	$O(n^3)$

Worked Examples – Computing Big-O

Three Nested Loops

```
FOR i FROM 0 TO n-1
  FOR j FROM 0 TO n-1
    FOR k FROM 0 TO n-1
      PRINT i, j, k
```

Basic operation: `PRINT`. Executes $n \times n \times n = n^3$ times. $\Rightarrow \mathbf{O(n^3)}$

Two Separate Loops

```
FOR i FROM 1 TO n: PRINT i
FOR j FROM 1 TO n^2: PRINT j
```

$O(n) + O(n^2) \Rightarrow$ drop lower: $\mathbf{O(n^2)}$

Two Nested Loops

```
sum = 0
FOR i FROM 0 TO n-1
  FOR j FROM 0 TO n-1
    sum = sum + 1
```

Basic operation: `sum = sum + 1`. Executes n^2 times. $\Rightarrow \mathbf{O(n^2)}$

Halving Loop

```
i = n
WHILE i > 1:
  PRINT i
  i = i / 2
```

i is halved each step: executes $\log_2 n$ times. $\Rightarrow \mathbf{O(\log n)}$

Triangular Loop

```
FOR i FROM 0 TO n-1
  FOR j FROM 0 TO i
    PRINT i, j
```

Inner loop runs $0 + 1 + 2 + \ldots + (n-1) = \frac{n(n-1)}{2}$ times. $\Rightarrow \mathbf{O(n^2)}$

Three Variables

```
FOR i FROM 0 TO n-1
  FOR j FROM 0 TO m-1
    FOR k FROM 0 TO p-1
      PRINT i, j, k
```

$\Rightarrow \mathbf{O(n \cdot m \cdot p)}$

Doubling While Loop with Inner For

```
i = 1
WHILE i < n:
  FOR j FROM 0 TO i:
    PRINT i, j
  i = i * 2
```

Outer loop: $\log_2 n$ iterations with $i = 1, 2, 4, \ldots$ Inner loop runs i times each iteration. Total: $\sum_{k=0}^{\log n} 2^k = 2n - 1 \approx n$. $\Rightarrow \mathbf{O(n)}$

Practical Implications of Complexity

Doubling the Input Size – What Happens?

Complexity	Effect of doubling n	Example
$O(1)$	No change	Hash table lookup
$O(\log n)$	$+1$ operation	Binary search
$O(n)$	$\times 2$ operations	Linear search
$O(n \log n)$	Slightly more than $\times 2$	Merge sort
$O(n^2)$	$\times 4$ operations	Bubble sort
$O(n^3)$	$\times 8$ operations	Naive matrix multiply
$O(2^n)$	Squares the time	Recursive Fibonacci

Scalability Example

If algorithm takes 1 minute for $n = 1000$ on old computer, new computer is $1000\times$ faster:

Complexity	Old max n in 1 min	New max n in 1 min
$T(n) = n$	1,000	1,000,000
$T(n) = n^3$	1,000	$\approx 10,000$
$T(n) = 10^n$	1,000	1,003

Lesson: a faster computer helps linear and polynomial algorithms much more than exponential ones.

Analysing the Function $f(n) = 3n^2 + 10n \log n + 1000n + 4 \log n + 9999$

The fastest-growing term is $3n^2$. All others are lower order.

$$f(n) = \Theta(n^2)$$

Abstraction and Decomposition in System Design

High-Level Requirements vs Low-Level Details

High-Level Abstraction

A **high-level requirement** describes *what* the system must do (capability/behaviour) without specifying *how* it is implemented internally. It focuses on the user's or stakeholder's perspective.

Example – Online Food Delivery System:

Requirement	Description	Stakeholders	Why High-Level	Challenge
User Authentication	Secure registration/login	Customers, Admins	Focuses on capability, not encryption details	Credential security
Payment Processing	Online and cash payment support	Customers, Restaurants	Hides gateway integration	PCI compliance
Real-time Tracking	Live order status updates	Customers, Delivery	No mention of GPS protocols	Latency, accuracy
Menu Management	Restaurants update menus	Restaurants, Admins	No DB schema specified	Concurrency/sync
Analytics Dashboard	Admin monitoring & reports	Admins	No query structure specified	Data aggregation scale

Why Abstraction is Useful in Early Design

- Reduces cognitive complexity – focus on what matters now
- Enables communication between technical and non-technical stakeholders
- Allows design changes without impacting other modules
- Supports modular, parallel development

Module Decomposition

A large system is broken into **modules**, each with:

- A clear **responsibility** (what it does)
- Defined **data** it handles
- Known **interactions** with other modules

Why decomposition matters for large systems:

- Parallel team development
- Independent testing and debugging
- Easier maintenance and future extension
- Avoids the "big ball of mud" problem (one huge, tangled component)

Pattern Recognition in Data

Identifying Patterns in Tabular Data

When given a dataset, look for:

- **Monotonic trends** – consistently increasing or decreasing values
- **Seasonal patterns** – periodic rises/falls
- **Outliers** – sudden drops or spikes (e.g. April dip)
- **Correlations** – do two columns rise together?

Prediction from Patterns

Given a linear trend, extrapolate: if values increase by $\approx \Delta$ per period, next value $\approx$ last value $+\Delta$.

Functional Decomposition of a System

Break a system's functions into sub-functions:

- **Patient Registration** → collect details, validate, store, assign ID
- **Book Appointment** → check availability, select slot, confirm, notify
- **Cancel Appointment** → verify identity, update schedule, notify doctor
- **Medical Records** → view history, add records, update records

Quick Reference – Key Formulae

Essential Formulae and Identities

$$\text{Sum of first } n \text{ integers:} \quad \sum_{i=1}^{n} i = \frac{n(n+1)}{2} \approx \frac{n^2}{2} \Rightarrow O(n^2)$$

$$\text{Triangular loop total:} \quad 0 + 1 + 2 + \cdots + (n-1) = \frac{n(n-1)}{2} \Rightarrow O(n^2)$$

$$\text{Geometric sum:} \quad 1 + 2 + 4 + \cdots + 2^k = 2^{k+1} - 1$$

$$\text{Logarithm identity:} \quad \log_a n = \frac{\log_b n}{\log_b a} \Rightarrow O(\log_a n) = O(\log n)$$

$$\text{Binary search steps:} \quad \lfloor \log_2 n \rfloor + 1$$

$$\text{Fibonacci (closed form):} \quad F_n \approx \frac{\varphi^n}{\sqrt{5}}, \quad \varphi = \frac{1+\sqrt{5}}{2} \approx 1.618$$

Summary Tables

Algorithm Comparison

Algorithm	Best	Average	Worst
Sequential Search	$O(1)$	$O(n)$	$O(n)$
Binary Search	$O(1)$	$O(\log n)$	$O(\log n)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Merge Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Fibonacci (recursive)	–	–	$O(2^n)$
Fibonacci (iterative)	–	–	$O(n)$

Recursion Type Summary

Type	Recursive Call Position	Typical Use
Head	First statement in body	Reverse operations
Tail	Last statement in body	Can be optimised as loop
Tree	Multiple calls per function	Fibonacci, tree traversal
Nested	Argument is a recursive call	Ackermann function
Indirect	Via another function	Mutual recursion

Big-O Complexity Quick Reference

Complexity	Name	n=10	n=100
$O(1)$	Constant	1	1
$O(\log n)$	Logarithmic	3	7
$O(n)$	Linear	10	100
$O(n \log n)$	Linearithmic	33	664
$O(n^2)$	Quadratic	100	10,000
$O(n^3)$	Cubic	1,000	1,000,000
$O(2^n)$	Exponential	1,024	$\approx 10^{30}$
$O(n!)$	Factorial	3,628,800	*astronomical*

These notes are compiled from SCS1304 lecture slides and lab sheets at UCSC.